알고리즘1 (2026-1)

6. 분할 정복 알고리즘 (2)

국립금오공과대학교 컴퓨터공학부

컴퓨터공학전공

김 경 수

목차

- 선택 문제 알고리즘
- 최근접 점 쌍 찾기
- 분할 정복 알고리즘 적용 시 유의할 점

학습목표

① 선택 문제의 의미를 파악하고, 분할 정복 알고리즘 패러다임을 이용하여 선택 문제를 해결하는 방법을 이해한다.

② 선택 문제 알고리즘의 시간 복잡도를 분석하고 유도할 수 있다.

③ 분할 정복 알고리즘의 활용이 유용한 경우와 유용하지 않은 경우를 이해하고, 주어진 문제가 분할 정복 알고리즘을 활용하여 해결하기 적합한 문제인지 분석할 수 있다.

선택 문제 알고리즘

선택 문제(Selection Problem)

- 선택 문제는 n개의 숫자들 중에서 k 번째로 작은 숫자를 찾는 문제
- k 번째로 작은 숫자를 찾는 간단한 방법
  - 방법 1: **최소 숫자를 k 번** 찾는다.
    - ➢ 단, 최소 숫자를 찾을 때 마다 해당 숫자를 입력에서 제거한다.
  - 방법 2: 숫자들을 **정렬한 후**, k번째 숫자를 찾는다.
  - 위의 "방법 1과 2"의 최악의 경우 시간 복잡도는 각각 $\boldsymbol{O(kn)}$과 $\boldsymbol{O(n \log n)}$이다.

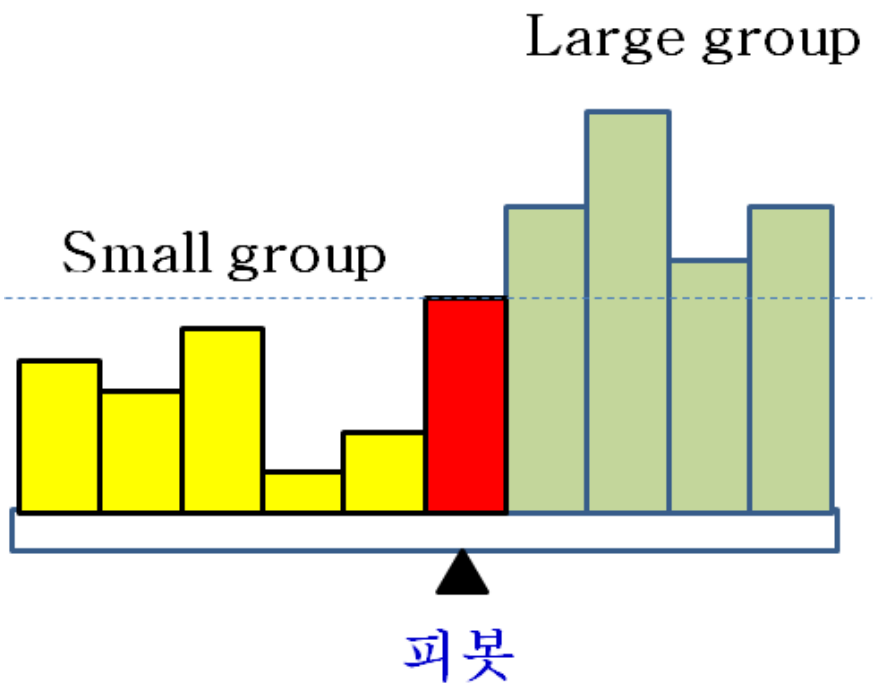
선택 문제의 아이디어

- 선택 문제도 "탐색 문제"의 일종이다.
- 이진 탐색
  - ➢ 정렬된 리스트에서 중간에 위치한 숫자와 탐색하는 숫자를 비교한 후, 리스트를 절반으로 나눠서 왼쪽 또는 오른쪽 부분 리스트만을 탐색함.

Reference: https://blog.naver.com/PostView.naver?isHttpsRedirect=true&blogId=jkssleeky&logNo=220715543451

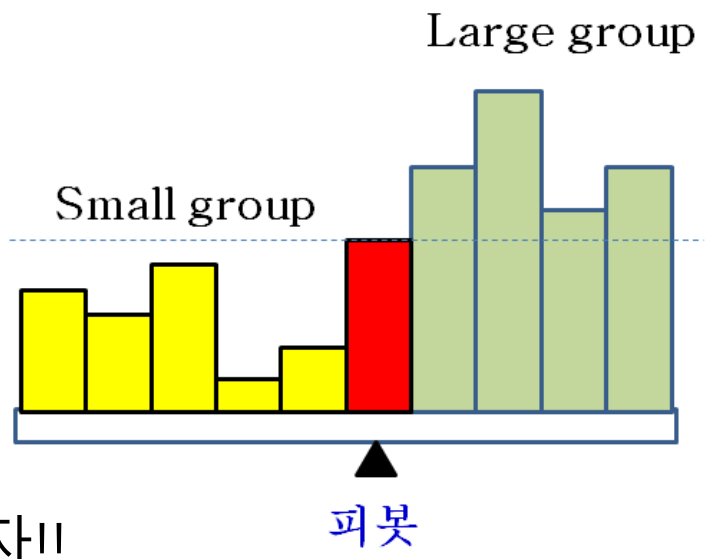
선택 문제의 아이디어

- k 번째로 작은 숫자를 찾는 효율적인 방법은?
  - ➢ 리스트의 값들이 정렬되어 있지 않으므로, 이진 탐색 알고리즘을 적용할 수 없다.
  - ➢ 이 경우에는 퀵 정렬과 유사하게 리스트 내에서 피봇을 선택한 후 피봇을 기준으로 리스트를 두 부분으로 분할한다.
    - ✓ **피봇의 왼쪽 → 피봇보다 작은 값**들이 위치
    - ✓ **피봇의 오른쪽 → 피봇보다 큰 값**들이 위치



선택 문제의 아이디어

- Small group은 피봇보다 작은 숫자의 그룹이고, Large group은 피봇보다 큰 숫자의 그룹이다.
- 이렇게 분할했을 때 알아야 할 것은 각 그룹의 크기, 즉, 각 그룹에 포함된 숫자의 개수이다.
  - ➢ 각 그룹의 크기를 알면,
  - ➢ k번째로 작은 숫자가 어느 그룹에 있는지를 알 수 있고,
  - ➢ 그 다음에는 그 그룹에서 몇 번째로 작은 숫자를 찾아야 하는지 알 수 있다.
- 이를 퀵 정렬에서 배운 "**피봇의 특성**"과 연관지어 생각해 보자!!



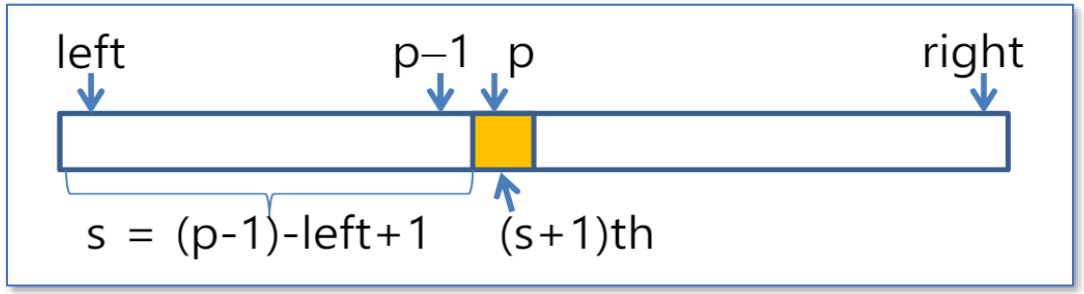
선택 문제의 아이디어

- Small group 내에 포함된 숫자의 개수를 **S**라 하자(**S = |Small group|**). 그렇다면…
- Small group에 k번째로 작은 숫자가 포함된 경우 (즉, k ≤ S)
  - ➢ k 번째로 작은 숫자를 **Small group**에서 찾는다.
- Large group에 k 번째로 작은 숫자가 포함된 경우 (즉, k > S)
  - ➢ **"k-S-1"** 번째로 작은 숫자를 **Large group**에서 찾아야 한다.

x_1	x_2	x_3	x_4	x_5	x_6	Pivot	x_7	x_8	x_9	x_{10}	x_{11}

Small group **Large group**

선택 문제 알고리즘




```
Algorithm Selection(A, left, right, k)

  1. p = Partition(A, left, right);

  2. S = (p-1)-left+1;                              // S = Small group의 크기

  3. if ( k <= S ) Selection(A, left, p-1, k);      // Small group에서 찾기

  4. else if ( k == S + 1 ) return A[p];            // pivot = k번째로 작은 숫자

  5. else Selection(A, p+1, right, k-S-1);          // Large group에서 찾기
```

선택 문제 알고리즘의 수행 과정

목표: 리스트 A[ ]에서 7번째로 작은 수(k=7)를 찾고자 한다.

Index	0	1	2	3	4	5	6	7	8	9	10	11
Value	**6**	**3**	**11**	**9**	**12**	**2**	**8**	**15**	**18**	**10**	**7**	**14**

- [시작] `Selection(A, 0, 11, 7)` 호출

Index	0	1	2	3	4	5	6	7	8	9	10	11
Value	**6**	**3**	**11**	**9**	**12**	**2**	**8**	**15**	**18**	**10**	**7**	**14**

- [Line 1] 선택된 피봇 “8”을 맨 왼쪽으로 이동

Index	0	1	2	3	4	5	6	7	8	9	10	11
Value	**8**	**3**	**11**	**9**	**12**	**2**	**6**	**15**	**18**	**10**	**7**	**14**

선택 문제 알고리즘의 수행 과정

```
In Selection(A, 0, 11, 7) :
```

- **[Line 1] 피봇 “8”을 기준으로 각각의 원소들의 위치를 이동** (※ 퀵 정렬과 동일)

Index	0	1	2	3	4	5	6	7	8	9	10	11
Value	8	3	11	9	12	2	6	15	18	10	7	14

Index	0	1	2	3	4	5	6	7	8	9	10	11
Value	8	3	7	6	2	12	9	15	18	10	11	14

- **[Line 1] 피봇 “8”을 원래 위치로 이동** (※ 퀵 정렬과 동일)

Index	0	1	2	3	4	5	6	7	8	9	10	11
Value	2	3	7	6	8	12	9	15	18	10	11	14

선택 문제 알고리즘의 수행 과정

In Selection(A, 0, 11, 7) :

0	1	2	3	4	5	6	7	8	9	10	11
2	3	7	6	8	12	9	15	18	10	11	14

- **[Line 2]** Small group의 크기를 계산

  S = (p-1)-left+1 = (4-1)-0+1 = 4

- **[Line 3, 4]** 현재 k = 7이고, S = 4 이므로 Line 3과 4의 조건문은 모두 False이다.

- **[Line 5]** **Selection(A,5,11,2)** 호출

  k=2, left=5, right=11 ➡ ∵ **"k-S-1"**번째로 작은 숫자를 **Large group**에서 찾아야 하기 때문

선택 문제 알고리즘의 수행 과정

In Selection(A, 5, 11, 2):

- **[Line 1]** 피봇 “14”를 선택한 후 피봇을 맨 왼쪽으로 이동

Index	5	6	7	8	9	10	11
Value	**12**	**9**	**15**	**18**	**10**	**11**	**14**

피봇 선택

Index	5	6	7	8	9	10	11
Value	**14**	**9**	**15**	**18**	**10**	**11**	**12**

피봇을 맨 왼쪽으로 이동

선택 문제 알고리즘의 수행 과정

```
In Selection(A, 5, 11, 2):
```

- **[Line 1] 피봇 “14”를 기준으로 각각의 원소들의 위치를 이동** (※ 퀵 정렬과 동일)

Index	5	6	7	8	9	10	11
Value	14	9	15	18	10	11	12

Index	5	6	7	8	9	10	11
Value	14	9	12	11	10	18	15

- **[Line 1] 피봇 “14”를 원래 위치로 이동** (※ 퀵 정렬과 동일)

Index	5	6	7	8	9	10	11
Value	10	9	12	11	14	18	15

선택 문제 알고리즘의 수행 과정

In Selection(A, 5, 11, 2):

5	6	7	8	9	10	11
10	9	12	11	14	18	15

- **[Line 2]** Small group의 크기 계산

  S = (p-1) - left+1 = (9-1)-5+1 = 4

- **[Line 3]** k = 2이므로, **Selection(A, 5, 8, 2)** 호출

  (∵ k=2, left=5, right=8)

선택 문제 알고리즘의 수행 과정

In Selection(A, 5, 8, 2):

- **[Line 1]** 피봇 “10”를 선택한 후 피봇을 맨 왼쪽으로 이동

  ※ 위와 같이 피봇이 10이 선택되면, 피봇을 기준으로 이미 정렬이 완료된 상태이다.

Index	5	6	7	8
Value	**10**	9	12	11

Index	5	6	7	8
Value	9	10	12	11

따라서 퀵 정렬과 같은 방법으로 피봇과 원소 9의 위치를 서로 교환(swap)한다.

선택 문제 알고리즘의 수행 과정

In Selection(A, 5, 8, 2):

- **[Line 2]** Small group의 크기를 계산

  S = (p-1)-left+1 = (6-1)-5+1 = 1

Index	5	6	7	8
Value	9	10	12	11

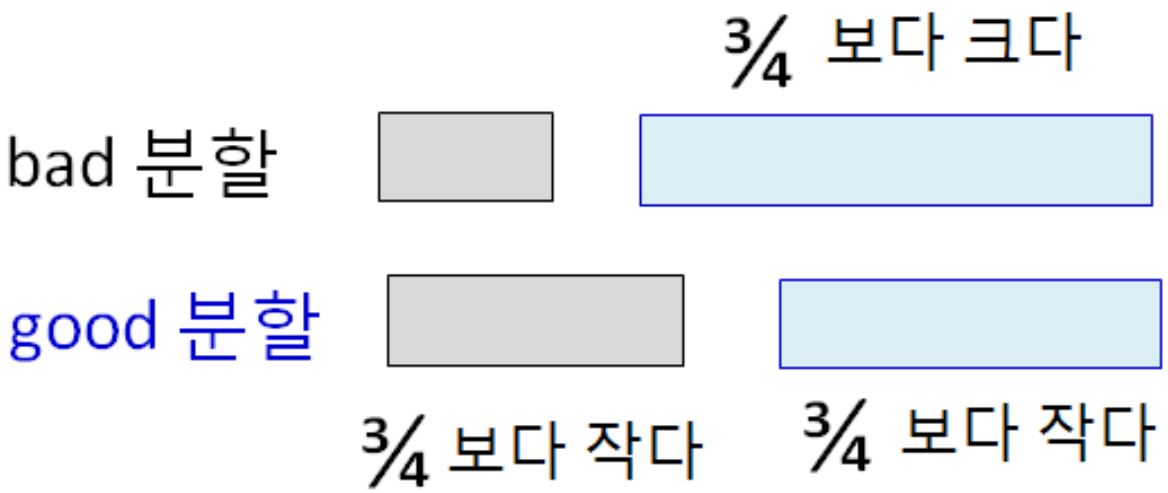
- **[Line 3]** if (k <= S)는 false이다. (∵ 현재 k=2 and S=1)
- **[Line 4]** if (k == S+1)은 true이므로, A[6]=10을 리턴한다.

선택 문제 알고리즘 고려 사항

- Selection 알고리즘은 분할 정복 알고리즘이며, 동시에 **랜덤(random) 알고리즘**이다.
  - ➢ 선택 알고리즘의 [Line 1]에서 피봇을 랜덤하게 정하기 때문이다.
- 피봇에 의해서 입력 리스트가 너무 한쪽으로 치우치게 분할되면 알고리즘의 효율성이 저하될 수 있다.
  - ➢ 즉, |Small group| << |Large group| 또는 |Small group| >> |Large group|일 때에는 알고리즘의 수행 시간이 증가한다.
- 선택 알고리즘이 호출될 때마다 입력 리스트가 한쪽으로 치우치게 분할될 확률은?
  - ➢ 동전을 던질 때 한쪽 면이 나오는 확률과 동일한 0.5이다.

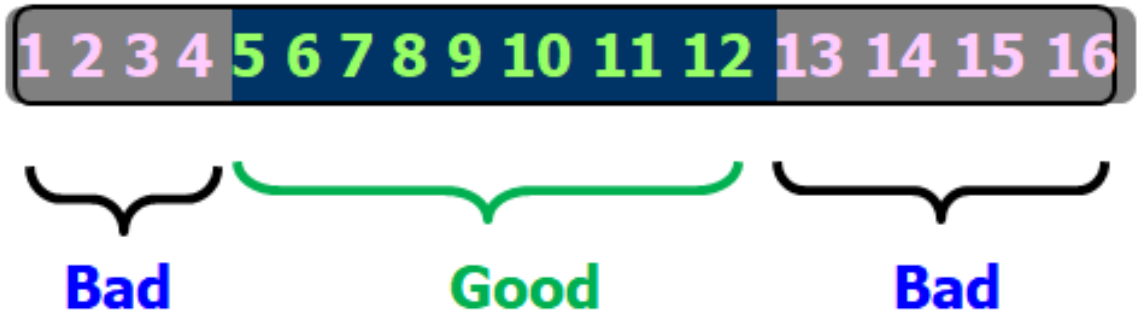
좋은(good) 분할과 나쁜(bad) 분할

- 분할된 두 그룹 중의 하나의 크기가 입력 크기의 **3/4**과 같거나 그 보다 크게 분할하면 **나쁜(bad) 분할**이라고 정의하자.
- 그렇다면, **좋은(good) 분할**은 그 반대의 경우로 아래와 같다.

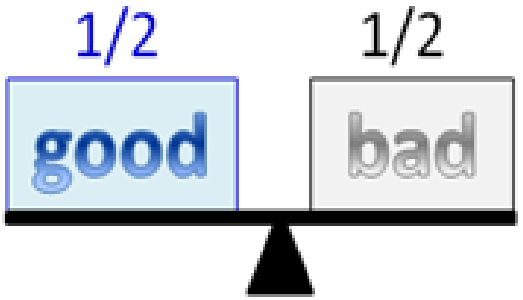


좋은(good) 분할과 나쁜(bad) 분할

- 다음과 같이 16개의 숫자가 주어졌다고 가정하자.



- 위의 리스트에서 good 분할이 되는 피봇을 선택할 확률과 bad 분할이 되는 피봇을 선택할 확률은 각각 0.5로 동일하다.

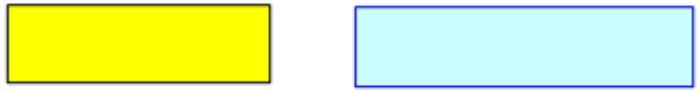


선택 문제 알고리즘의 시간 복잡도

- 피봇을 랜덤하게 선택했을 때 good 분할이 될 확률이 0.5이므로, 평균 2회 연속해서 랜덤하게 피봇을 정하면 good 분할을 수행할 수 있다.
- 이 경우, 매 2회 호출마다 good 분할이 이루어지기 때문에 good 분할만 연속하여 이루어졌을 때의 시간 복잡도를 구한 후 그 값에 2를 곱하여 평균 경우의 시간 복잡도를 계산할 수 있다.

좋은 분할이 연속적으로 수행되는 경우의 예

1번째 good 분할



$\frac{3}{4}$ 보다 작다 $\frac{3}{4}$ 보다 작다

2 번째 good 분할

$\left(\frac{3}{4}\right)^2$ 보다 작다

3 번째 good 분할

$\left(\frac{3}{4}\right)^3$ 보다 작다

⋮

i 번째 good 분할

$\left(\frac{3}{4}\right)^i$ 보다 작다

평균 경우의 시간 복잡도 분석

- 입력의 크기가 n 에서부터 3/4배로 연속적으로 감소되며, 입력 크기가 1인 경우에 분할이 종료된다.
- i 번째 분할까지의 과정을 분석하면 다음과 같다.

$$
\begin{aligned}
& n + \frac{3}{4}n + \left(\frac{3}{4}\right)^2 n + \left(\frac{3}{4}\right)^3 n + \cdots + \left(\frac{3}{4}\right)^{i-1} n + \left(\frac{3}{4}\right)^i n \\
&= n\left[1 + \frac{3}{4} + \left(\frac{3}{4}\right)^2 + \left(\frac{3}{4}\right)^3 + \cdots + \left(\frac{3}{4}\right)^{i-1} + \left(\frac{3}{4}\right)^i\right] \\
&\cong \boldsymbol{O(n)}
\end{aligned}
$$

- 따라서 평균 경우의 시간 복잡도는 $\boldsymbol{2 \times O(n) = O(n)}$이다.

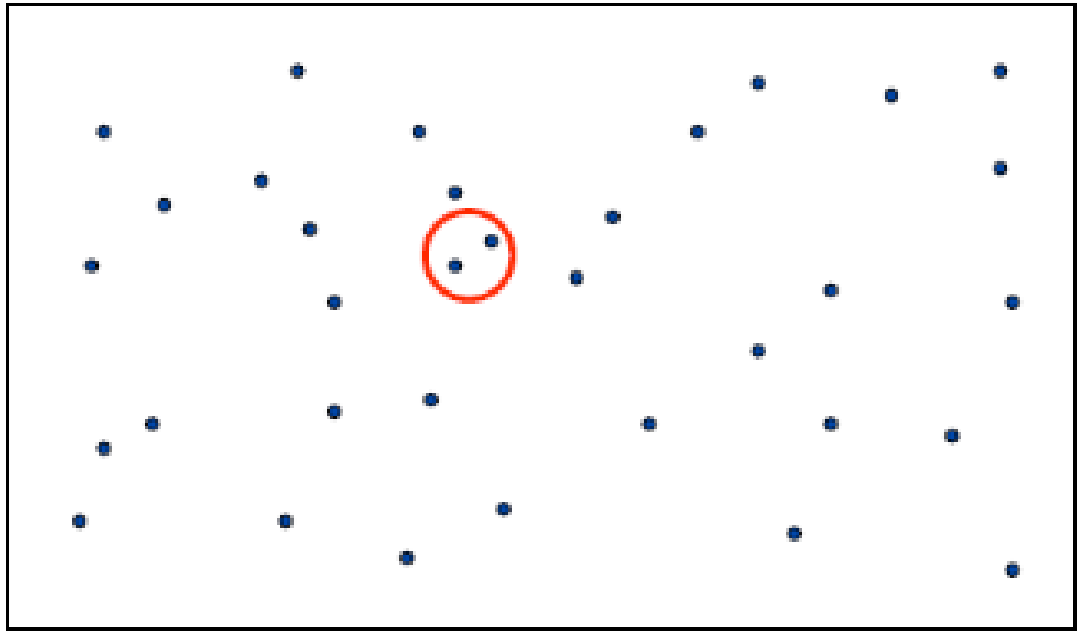
선택 문제 알고리즘의 응용

- 선택 알고리즘은 데이터 분석을 위한 **중앙값(median)**을 찾는데 활용
  - ➢ 데이터 분석에서는 주어진 자료들의 대표 값으로 평균(average)도 유용하게 사용되지만, 중앙값(median)이 더욱 설득력이 높은 경우가 많다.
  - ➢ 예를 들어, 대부분의 데이터가 1이고, 오직 1개의 숫자가 매우 큰 숫자(e.g., noise 또는 outlier value)이면, 평균값은 매우 왜곡된 분석 결과가 되어 집단을 대표할 수 없다.

최근접 점 쌍 찾기

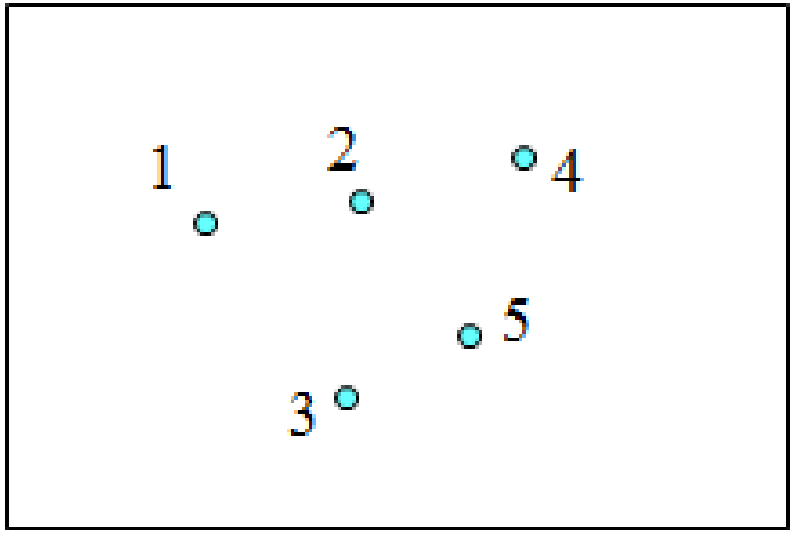
최근접 점 쌍 찾기

- 최근접 점의 쌍(Closest Pair) 찾기 문제는 2차원 평면상의 n개의 점이 입력으로 주어질 때, 거리가 가장 가까운 한 쌍의 점을 찾는 문제이다.



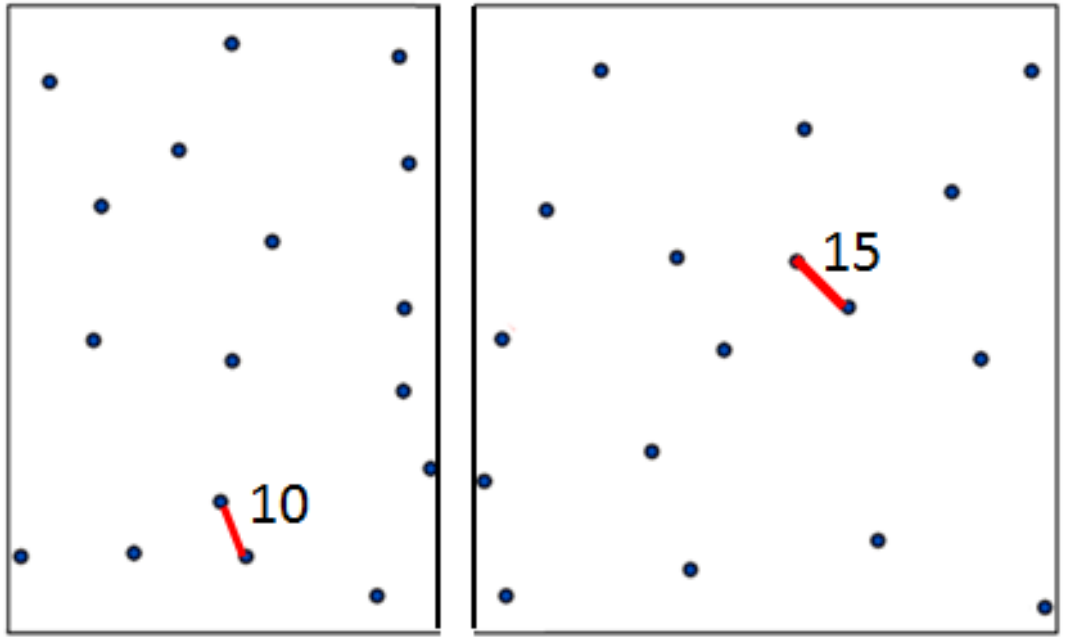
최근접 점 쌍 찾기

- 거리가 가장 가까운 한 쌍의 점을 찾는 간단한 방법
  - ➢ 모든 점 각각에 대하여 두 점 사이의 거리를 계산한 후, 가장 가까운 점의 쌍을 찾는다.
  - ➢ 예를 들어, 5개의 점이 아래와 같이 주어지면, 1-2, 1-3, 1-4, 1-5, 2-3, 2-4, 2-5, 3-4, 3-5, 4-5 사이의 거리를 각각 계산하여 그 중에 최소 거리를 가진 쌍이 최근접 점의 쌍이 된다.
  - ➢ 이 때 비교해야 할 쌍은 몇 개이며, 시간 복잡도는 얼마인가?
    - ✓ **${}_nC_2$ = n(n-1)/2**
    - ✓ n=5이면, 5(5-1)/2 = 10
    - ✓ n개의 점에 대한 수행 시간 복잡도는 $O(n^2)$이다.
    - ✓ 한 쌍의 거리 계산을 위한 시간 복잡도는 $O(1)$이다.
    - ✓ 따라서, 총 시간 복잡도는 $O(n^2) \times O(1) = O(n^2)$이다.



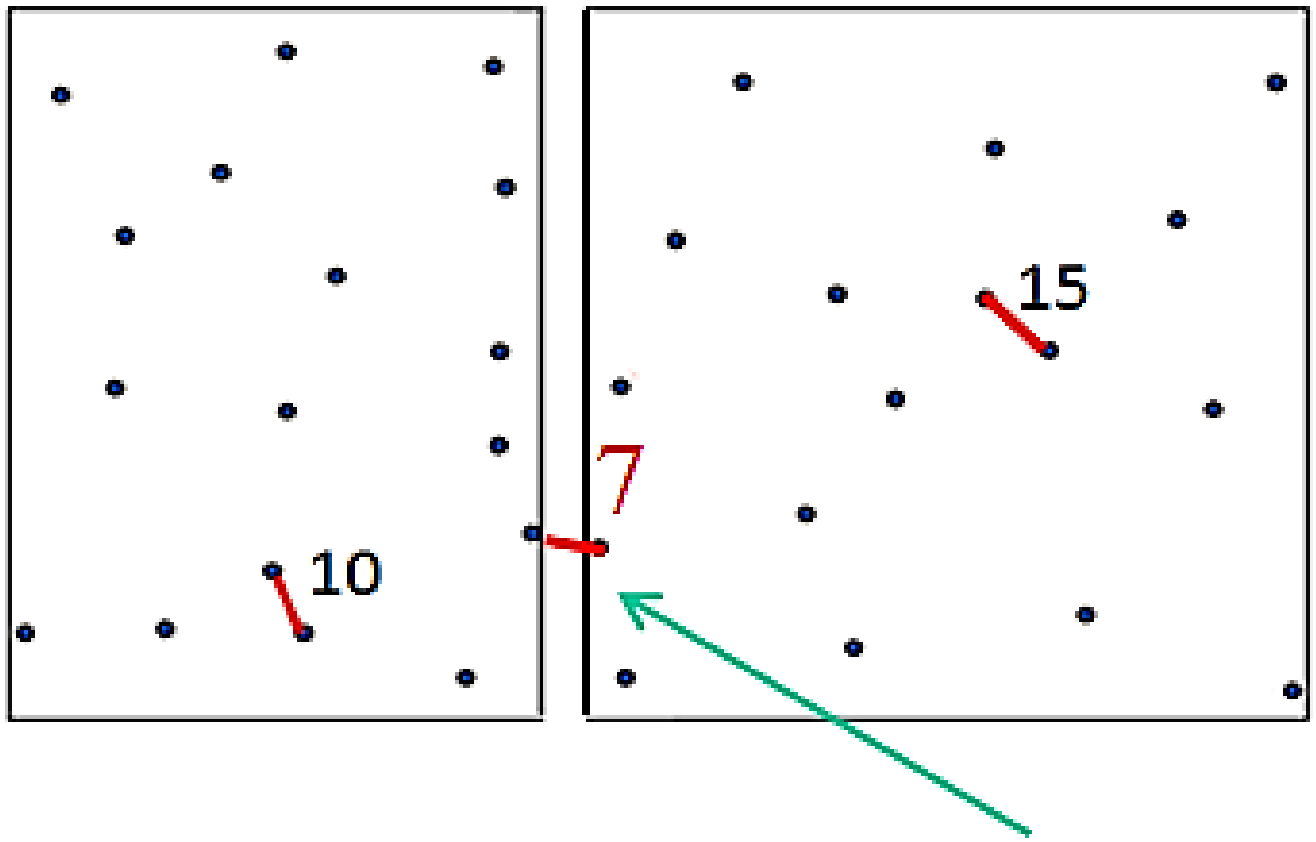
최근접 점 쌍 찾기

- $O(n^2)$보다 더욱 효율적인 방법은? → **분할 정복 알고리즘을 이용**
  - n개의 점을 **½개로 분할**하여 각각의 부분 문제에서 최근접 점의 쌍을 찾는다.
  - 그 다음에, 2개의 부분 해 중에서 짧은 거리를 가진 점의 쌍을 찾는다.



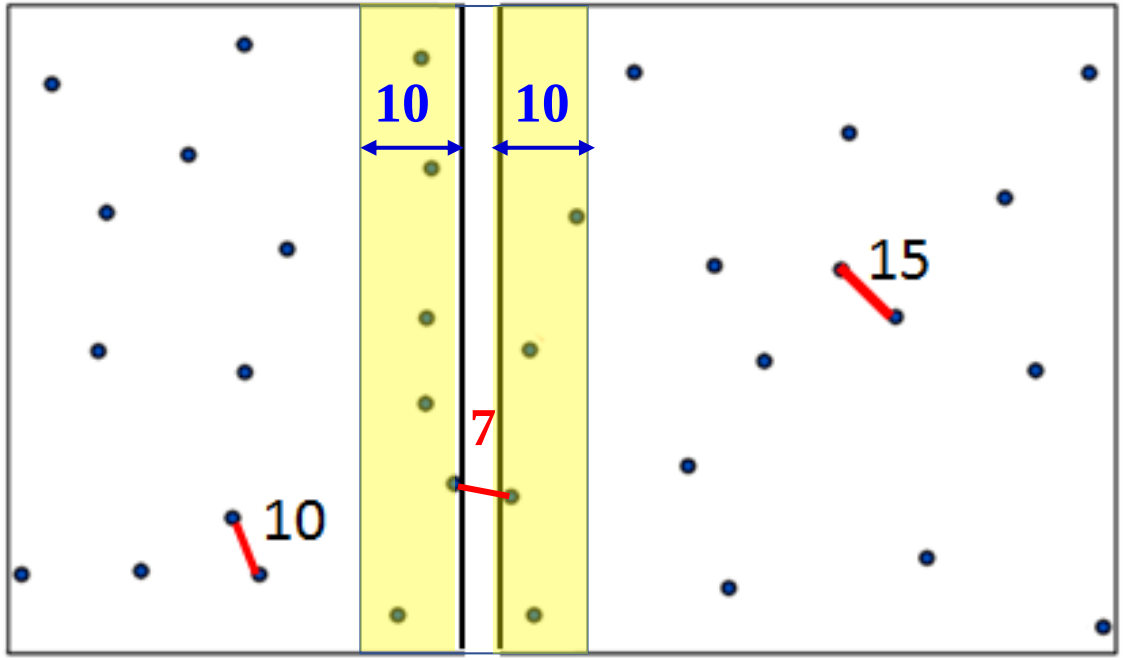
최근접 점 쌍 찾기

➢ 단, 두 부분 문제를 하나로 취합할 때는 중간 영역을 고려해야 한다.



최근접 점 쌍 찾기 – 중간 영역 안의 점들 확인하기

➢ 이를 위해, 기존에 두 부분 문제에서 발견된 최근접 쌍의 거리인 d 이내의 중간 영역 안에 포함된 점들 중 거리가 더욱 짧은 근접 쌍이 존재하는지 확인한다.



✓ 위 예제에서는 10과 15 중에서 짧은 거리인 10 이내의 중간 영역 안에 포함된 점들 중에 더 근접한 점의 쌍이 있는지 확인해야 한다.

최근접 점 쌍 찾기 – 중간 영역 안의 점들 확인하기

- $d = min\{$ 왼쪽 부분의 최근접 점 쌍 사이의 거리,
  오른쪽 부분의 최근접 점 쌍 사이의 거리 $\}$
- 각각의 점 (x, y)는 1차원 배열 내에 x-좌표의 오름차순으로 정렬되어 저장된다.

(※ 아래의 배열에는 x-좌표만 표현되어 있으나 실제 구현시에는 y-좌표도 포함되어야 함.)

왼쪽 부분문제의 가장 오른쪽 점

오른쪽 부분문제의 가장 왼쪽 점

0	1	2	3	4	5	6	7	8	9
(1, -)	(13, -)	(17, -)	(25, -)	(26, -)	(28, -)	(30, -)	(37, -)	(45, -)	(56, -)

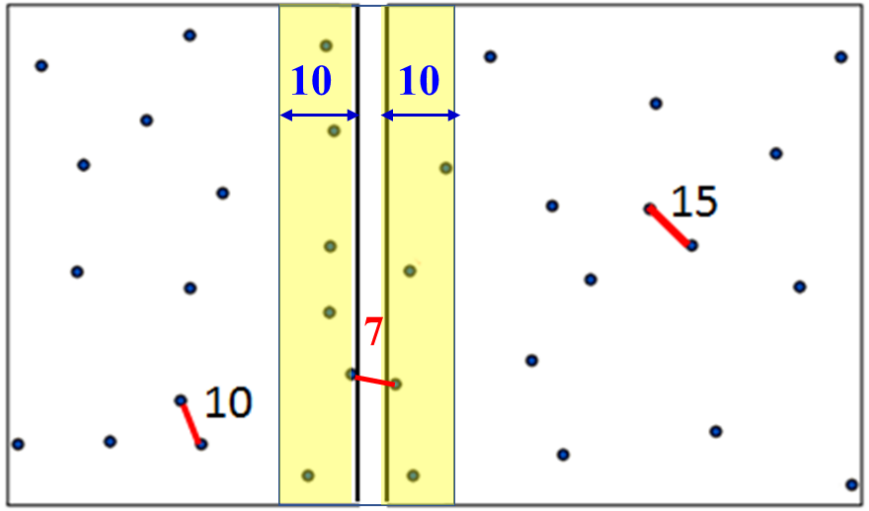
< 2차원 좌표상의 점들을 표현하기 위한 자료구조 예 >

최근접 점 쌍 찾기 – 중간 영역 안의 점들 확인하기

- 중간 영역에 속한 점 =

  {왼쪽 부분 문제에서 가장 오른쪽 점의 x-좌표에서 d를 뺀 값과 오른쪽 부분 문제에서 가장 왼쪽 점의 x-좌표에 d를 더한 값 사이의 x-좌표 값을 가진 모든 점들}

- $d = 10$이면, 점 (25, -), (26, -), (28, -), (30, -), (37, -)이 중간 영역에 속한다.



- 2차원 좌표상의 점들을 표현하기 위한 자료구조

d = 10

0	1	2	3	4	5	6	7	8	9
(1,–)	(13,–)	(17,–)	(25,–)	(26,–)	(28,–)	(30,–)	(37,–)	(45,–)	(56,–)

26–d = 16　　28+d = 38

최근접 점 쌍 찾기 알고리즘

Algorithm ClosestPair(S)

- 입력: 2차원 평면상의 점들을 x-좌표의 오름차순으로 정렬하여 저장한 1차원 배열 S
- 출력: 배열 S에 포함되어 있는 점들 중 최근접 점의 쌍(pair)

1. **if** (|S| <= 3) **return** "2 또는 3개의 점들 중 거리가 가장 짧은 점들의 쌍";
2. 정렬된 S를 같은 길이의 S_L과 S_R로 분할한다. (※ 단, |S|가 홀수이면, $|S_L|=|S_R|+1$이 되도록 분할)
3. CP_L = **ClosestPair**(S_L); // CP_L: S_L에서의 최근접 점의 쌍 (p_i, p_j)
4. CP_R = **ClosestPair**(S_R); // CP_R: S_R에서의 최근접 점의 쌍 (p_i, p_j)
5. d = **min**{ **dist**(CP_L), **dist**(CP_R) };
6. CP_C = 중간 영역에 속하는 최근접 점의 쌍;
7. **return** (CP_L, CP_C, CP_R 중 거리가 가장 짧은 쌍);

- **min(a,b):** 두 수 a와 b 중에서 최소값
- **dist(CP_L):** 최근접 쌍 CP_L의 두 점 간의 거리(distance) (※ 일반적으로 Euclidean distance를 사용)

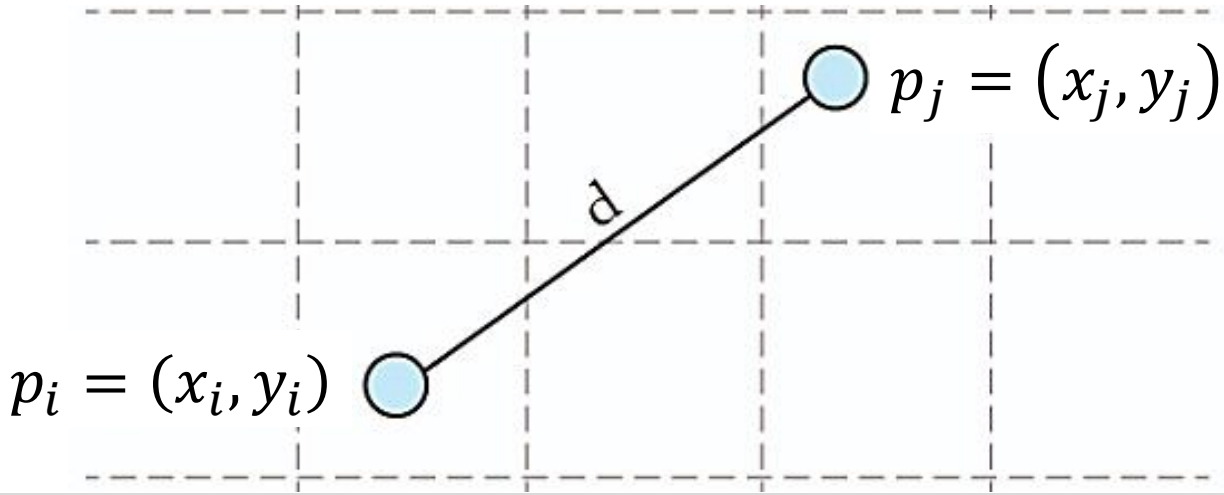
[참고] 2차원 공간에서 두 점 사이의 거리 계산

- **Euclidean distance (유클리드 거리)**
  - ➢ 임의의 두 점이 2차원 공간에서 정의될 때, $p_i = (x_i, y_i)$이고, $p_j = (x_j, y_j)$.
  - ➢ 이때, p_i와 p_j 사이의 거리는 다음과 같이 계산됨.

$$dist(p_i, p_j) = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}$$

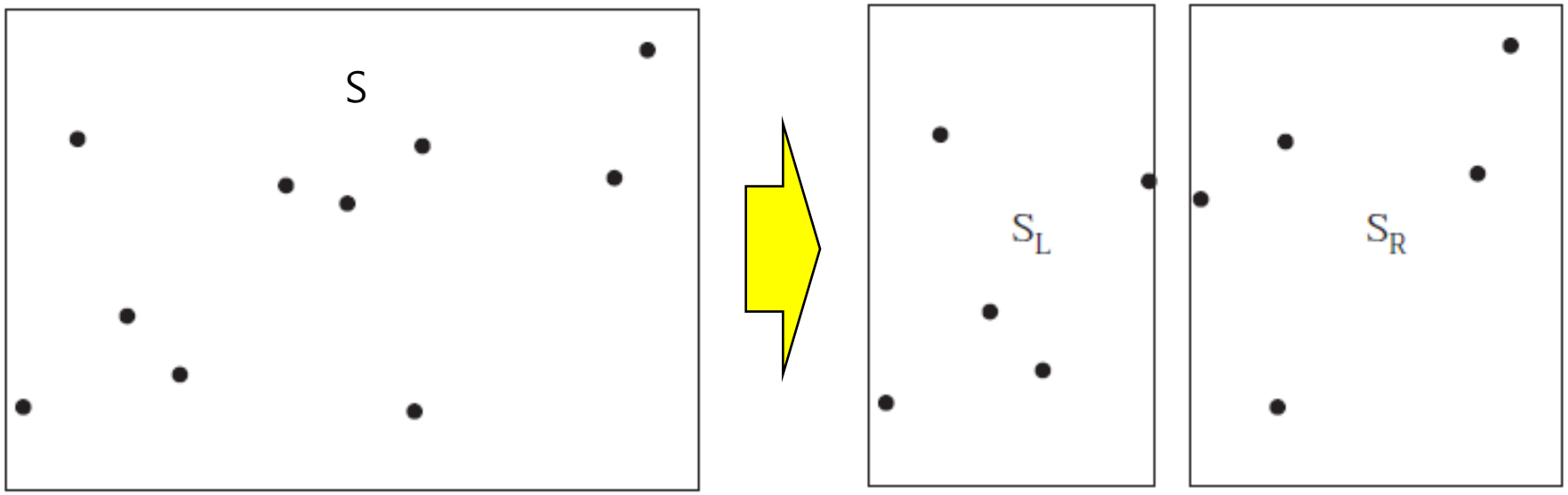
➢Euclidean distance 공식 유도 방법

✓ 피타고라스의 정리로부터 유도 가능

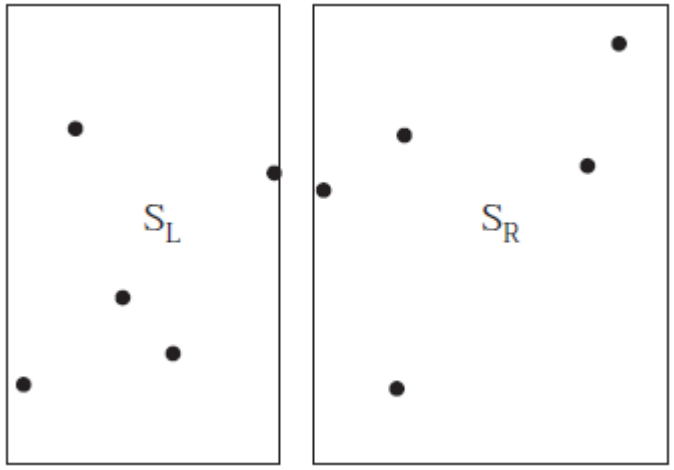


최근접 점 쌍 찾기 알고리즘의 수행 과정

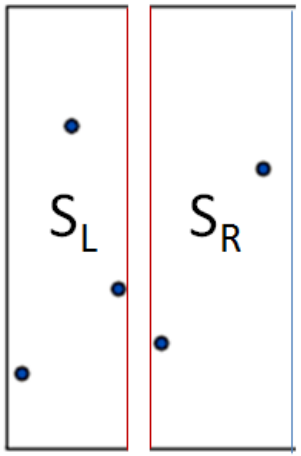
- **ClosestPair(S) 호출 ①**
  - ➢ Line 1: S에 포함된 점의 수가 3 이상이므로 line 2를 수행한다.
  - ➢ Line 2: S를 S_L과 S_R로 분할한다.



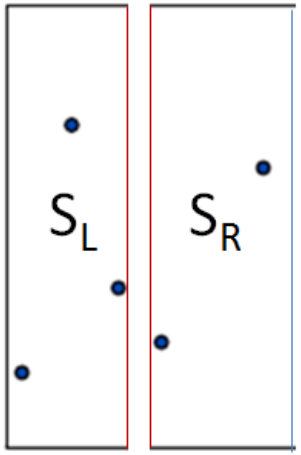
최근접 점 쌍 찾기 알고리즘의 수행 과정



Line 3: ClosestPair(S_L) **호출** ②



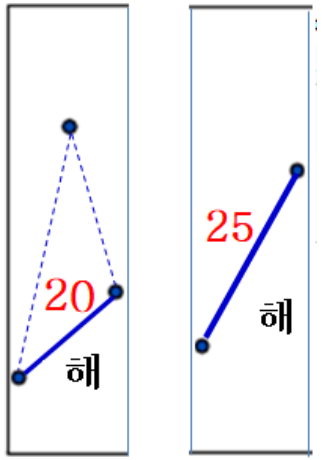
최근접 점 쌍 찾기 알고리즘의 수행 과정



ClosestPair(S_L) **호출**

ClosestPair(S_R) **호출**

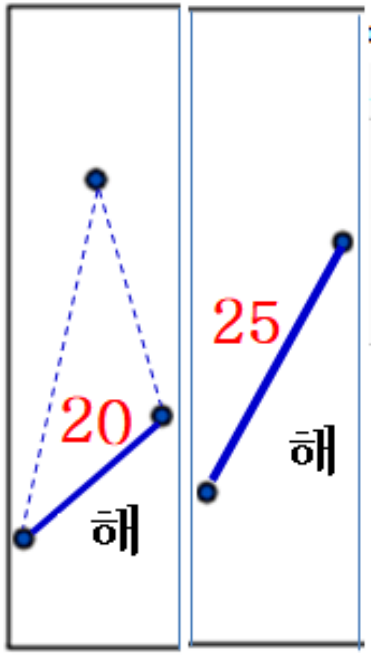
S_L에 포함되어 있는 점의 개수가 3 이하이므로, Line 1을 수행한다.

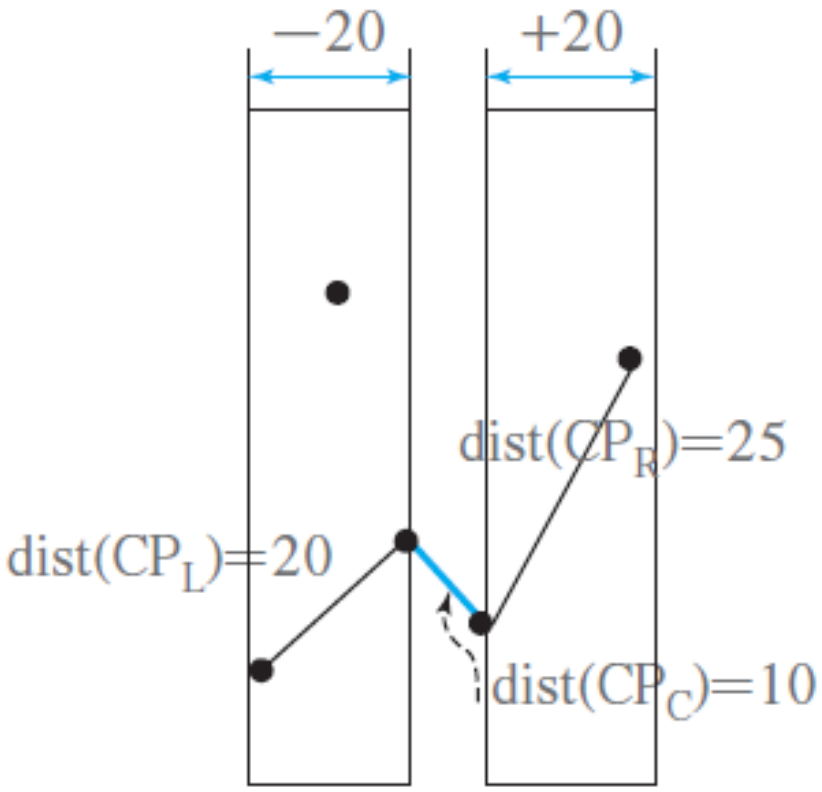


S_R에 포함되어 있는 점의 개수가 3 이하이므로, Line 1을 수행한다.

최근접 점 쌍 찾기 알고리즘의 수행 과정

- ②의 ClosestPair(S_L) 호출에서 line 3~4가 수행되었다.
- 이제 line 5-6을 수행하여, 중간 영역에 속하는 최근접 점 쌍 CP_C 를 찾는다.

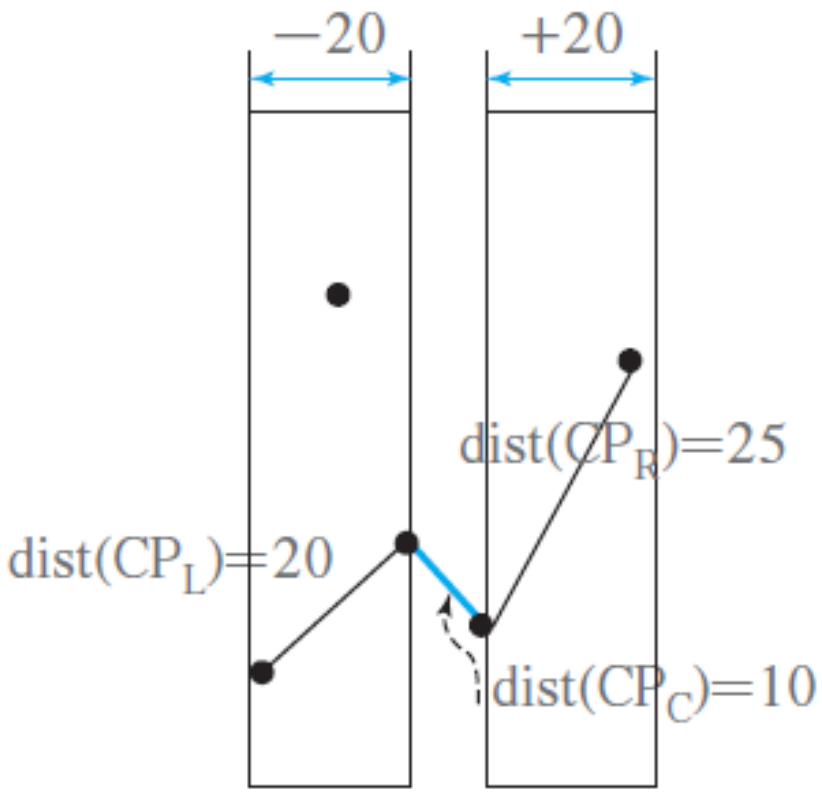




$d = \min\{dist(CP_L), dist(CP_R)\}$

※ 중간 영역에 속하는 최근접 점을 찾는 효율적인 방법은 이후 슬라이드(pp. 44 – 45) 참조.

최근접 점 쌍 찾기 알고리즘의 수행 과정



- ①의 ClosestPair(S) 호출에서 line 3:

  ClosestPair(S_L) 호출

- 상기 ClosestPair(S_L)의 호출 결과로 10을 반환

  dist(CP_L) = 20

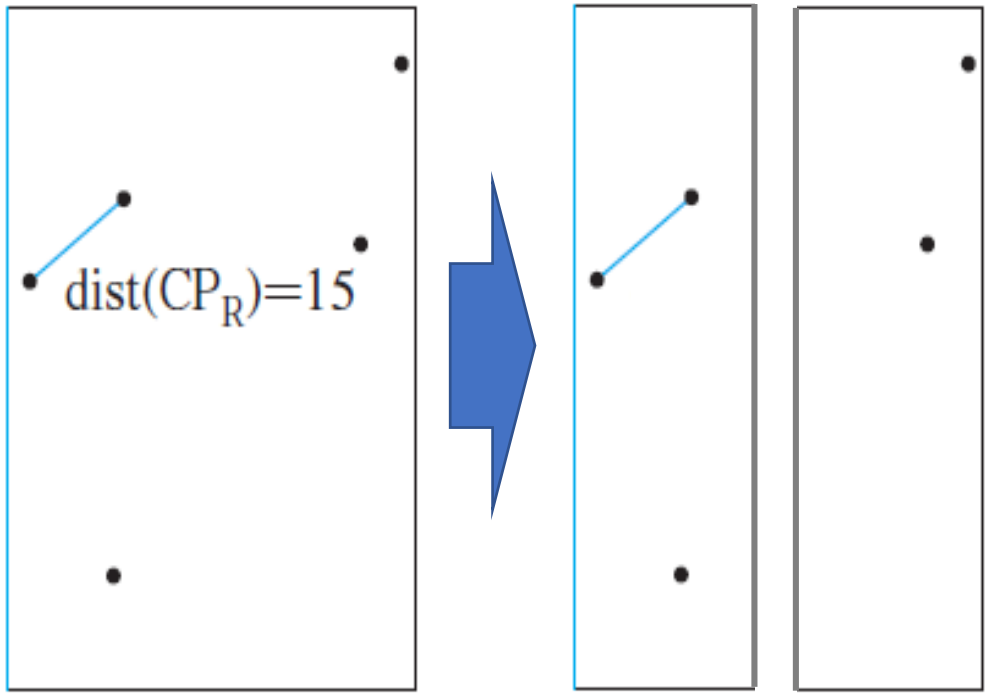
  dist(CP_R) = 25

  dist(CP_C) = 10

  ↓

  return (CP_L, CP_C, CP_R 중 거리가 가장 짧은 쌍)

최근접 점 쌍 찾기 알고리즘의 수행 과정



- ①의 ClosestPair(S) 호출 당시 line 4:

  ClosestPair(S_R) 호출

- 상기 ClosestPair(S_R)의 호출 결과로 15를 반환한다.

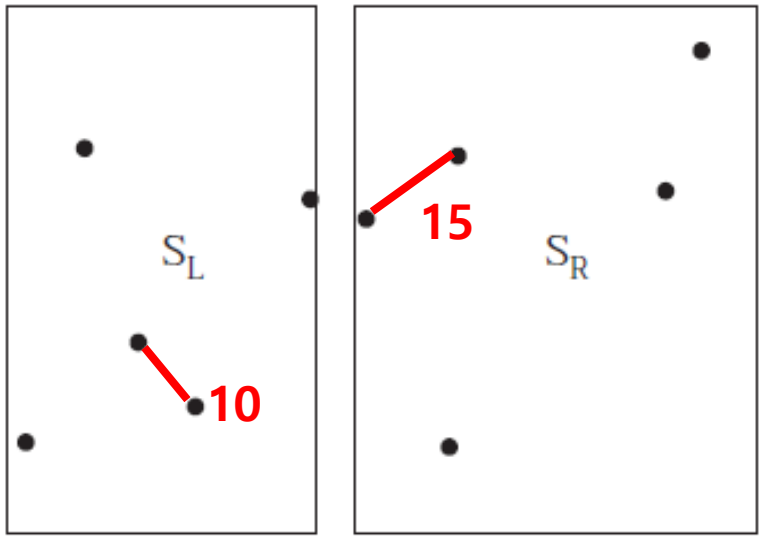
$$\because \min\{\mathrm{dist}(CP_L), \mathrm{dist}(CP_R), \mathrm{dist}(CP_C)\}$$

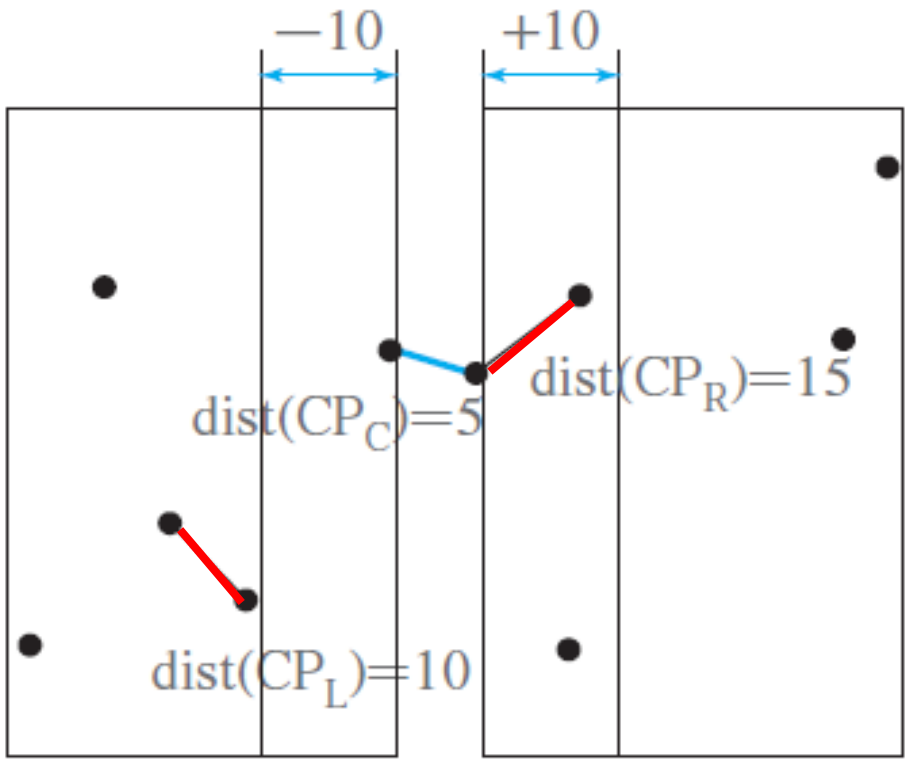
$$= \mathrm{dist}(CP_R)$$

$$= 15$$

최근접 점 쌍 찾기 알고리즘의 수행 과정

- Line 5: line 3~4에서 찾은 최근접 점 쌍 사이의 거리인 dist(CP_L)=10과 dist(CP_R)=15 중에서 작은 값을 d로 설정한다.
- Line 6: 중간 영역에 있는 점들 중에서 CP_C를 찾는다.

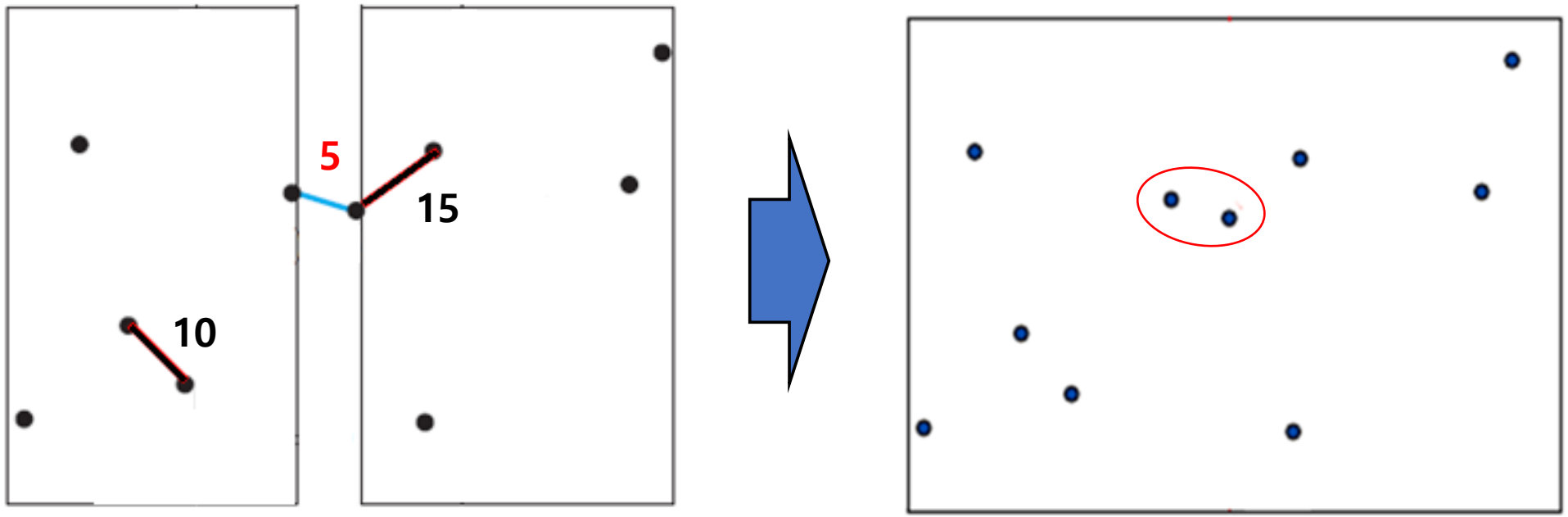




d = **min**{**dist**(CP_L), **dist**(CP_R)}

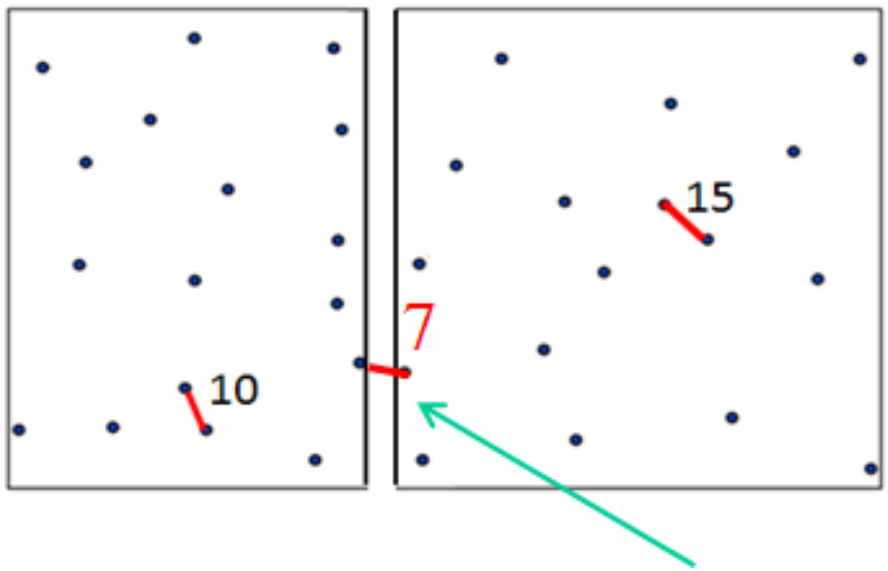
최근접 점 쌍 찾기 알고리즘의 수행 과정

- Line 7: dist(CP_L)=10, dist(CP_C)=5, dist(CP_R)=15 중에서 가장 거리가 짧은 쌍인 CP_C를 최근접 쌍의 점으로 최종적으로 리턴한다.



중간 영역에서 CP_C 를 찾는 효율적인 방법

- 왼쪽 영역을 S_L, 오른쪽 영역을 S_R이라 할 때, S_L과 S_R에 양 점이 포함된 쌍들의 거리를 효과적으로 계산하는 방법은?
  - ➢ 총 n개의 점 중에서 중간 영역에 포함될 수 있는 점의 최대 개수는 n개이다.
  - ➢ 이때, Brute force 방법으로 중간 영역 내 모든 점 사이의 거리를 계산하는데 소요되는 시간 복잡도는 $O(n^2)$이다.
  - ➢ 이는 상당히 비효율적인 방법이다.



- 보다 효율적인 방법은 무엇인가?
  - ➢ 중간 영역의 점들을 **y축을 기준으로 오름차순 또는 내림차순으로 정렬**한다.

중간 영역에서 CP_C 를 찾는 효율적인 방법

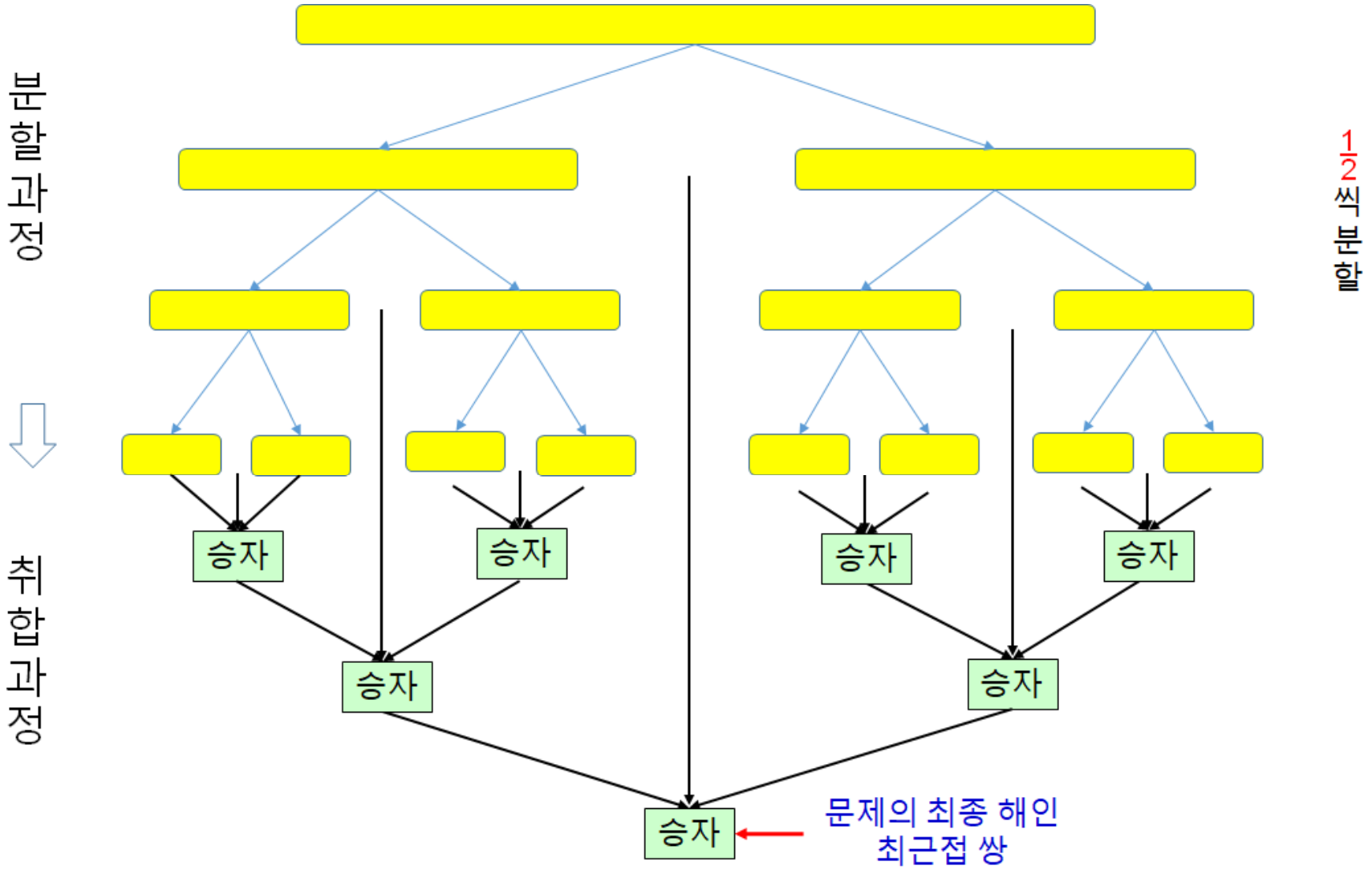
- 중간 영역에 포함되는 점들을 **y축을 기준으로 내림차순으로 정렬**한 점들을 순차적으로 $p_1, p_2, \ldots, p_n$이라 하자. (※ 단, $\forall i$ s.t $1 \le i \le n$, $p_i = (x_i, y_i)$)
- 임의의 두 점 p_i와 p_j 사이의 거리는 Euclidean distance로 계산된다.

$$\mathrm{Dist}(p_i, p_j) = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}$$

- 이때, 모든 점들이 **y축을 기준으로 정렬된 상태**이므로, 점 p_i에 대해서 다른 점들 간의 거리를 아래와 같이 효율적으로 계산할 수 있다.

```
for i = 1 to n-1 do
    for j = i+1 to n do
        if y_j - y_i >= d then break;
        d_ij = dist(p_i, p_j);
        if d_ij < d then d = d_ij;
```

최근접 점 쌍 찾기 알고리즘의 분할 정복 개념

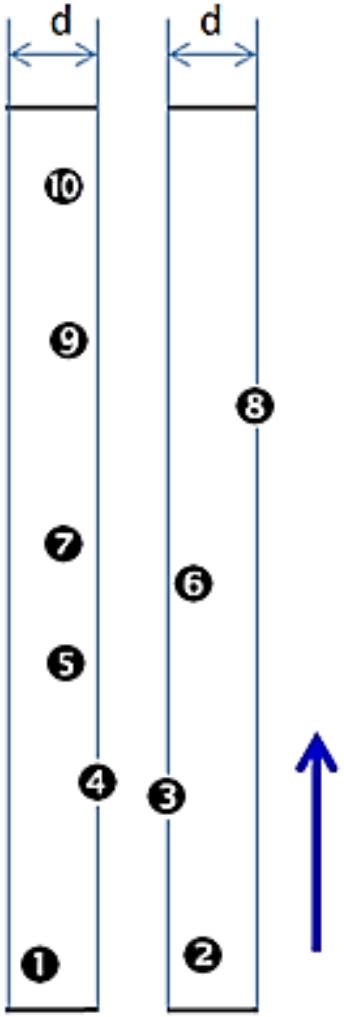


최근접 점 쌍 찾기 알고리즘의 시간 복잡도 분석

- 배열 S에 총 n개의 점이 있으면 전처리(preprocessing) 과정으로 S의 점을 **x-좌표를 기준으로 정렬**을 수행함 → $\boldsymbol{O(n \log n)}$
- [Line 1] 배열 S에 3개의 점이 있는 경우 총 3번의 거리 계산이 필요하고, S 내의 점의 수가 2이면, 1번의 거리 계산이 필요함. → $\boldsymbol{O(1)}$
- [Line 2] 정렬된 배열 S를 S_L과 S_R로 분할 시 S는 이미 정렬되어 있으므로, 배열의 중간 인덱스를 기준으로 분할함. → $\boldsymbol{O(1)}$
- [Line 3~4] S_L과 S_R에 대하여 각각 ClosestPair를 순환 호출하여 반복적으로 분할을 수행함. 이 때, 순환 호출 과정은 합병 정렬과 동일함.

최근접 점 쌍 찾기 알고리즘의 시간 복잡도 분석

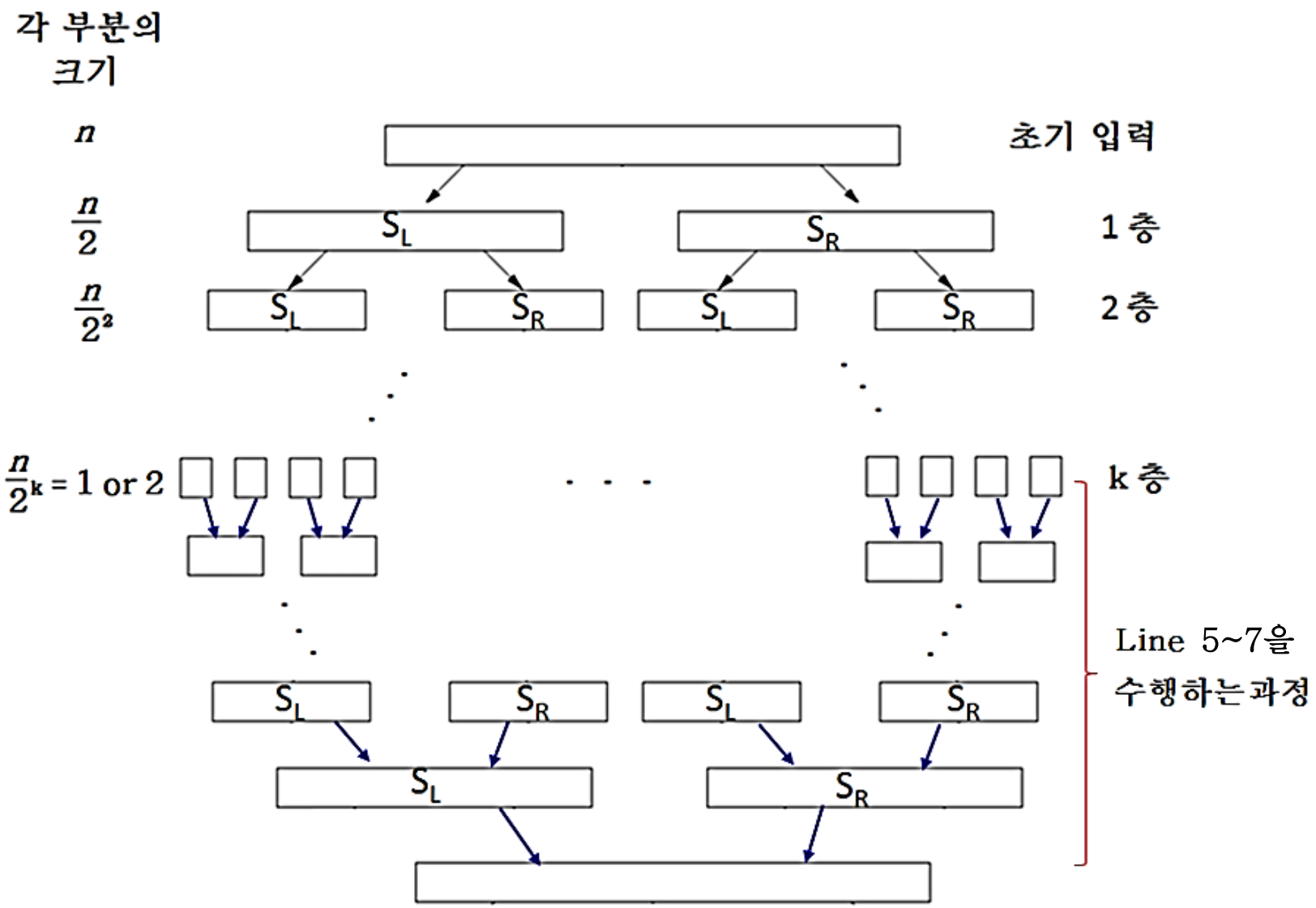
- [Line 5~6]
  - ➢ d = min{dist(CP_L), dist(CP_R)}일 때 중간 영역에 속하는 점들 중에서 최근접 점의 쌍을 찾음.
  - ➢ 이를 위해 먼저 중간 영역에 있는 점들을 **y-좌표를 기준으로 정렬**한 후에, 아래에서 위로(또는 위에서 아래로) 각 점을 기준으로 거리가 d이내인 주변의 점들 사이의 거리를 각각 계산하며, 이 영역에 속한 점들 중에서 최근접 점의 쌍을 찾음.
  - ➢ y-좌표를 기준으로 정렬하는데 $\boldsymbol{O(n \log n)}$의 시간이 걸리고, 아래에서 위로 올라가며 각 점에서 주변의 점들 사이의 거리를 계산하는데 **O(1)** 시간이 소요됨. (※ 왜냐하면 각 점과의 거리를 계산해야 하는 이웃 점들의 수가 **O(1)** 개이기 때문.)



최근접 점 쌍 찾기 알고리즘의 시간 복잡도 분석

- [Line 7] 3개의 점의 쌍 중에 가장 짧은 거리를 가진 점의 쌍을 리턴 → $\boldsymbol{O(1)}$
- ClosestPair 알고리즘의 분할과정은 합병 정렬의 분할 과정과 동일
  - ➢ 그러나 ClosestPair 알고리즘에서는 해를 취합하여 올라가는 과정인 line 5~7에서 $\boldsymbol{O(n \log n)}$ 시간이 필요함.
- k 레벨까지 분할된 후, 각 레벨별로 line 5~7이 수행됨.
  - ➢ 이 때, 각 레벨에서의 시간 복잡도는 $\boldsymbol{O(n \log n)}$.
  - ➢ 위 복잡도에 총 레벨의 개수인 $\boldsymbol{\log n}$ 을 곱하면 수행시간은 $\boldsymbol{\log n \times O(n \log n) = O(n \log^2 n)}$이다.

최근접 점 쌍 찾기 알고리즘의 시간 복잡도 분석



점화식과 반복 대치를 이용한 시간 복잡도 분석

- 최근접 점 쌍 찾기 문제를 점화식으로 나타내면 다음과 같다.

$$\boldsymbol{T(n) = 2T\left(\frac{n}{2}\right) + cn\log n} \text{ (※ 단, } T(1) = 0\text{이고 } c\text{는 상수)}$$

- $n = 2^k$이고, 더 이상 분할할 수 없을 때 까지 위 점화식을 반복 대치 방법으로 풀어보면 다음과 같다.

$$\begin{aligned}
T(n) &= 2T\left(\frac{n}{2}\right) + cn\log n \\
&= 2\left(2T\left(\frac{n}{4}\right) + c\frac{n}{2}\log\frac{n}{2}\right) + cn\log n \\
&= 2\left(2\left(2T\left(\frac{n}{8}\right) + c\frac{n}{4}\log\frac{n}{4}\right) + c\frac{n}{2}\log\frac{n}{2}\right) + cn\log n \\
&= 2\left(2\left(2\left(2T\left(\frac{n}{16}\right) + c\frac{n}{8}\log\frac{n}{8}\right) + c\frac{n}{4}\log\frac{n}{4}\right) + c\frac{n}{2}\log\frac{n}{2}\right) + cn\log n \\
&= \dots \\
&= 2^kT\left(\frac{n}{2^k}\right) + cn\left(\log\frac{n}{2^{k-1}} + \log\frac{n}{2^{k-2}} + \cdots + \log\frac{n}{2^0}\right) = cn\left(k\log n - \log 2 - \log 4 - \cdots - \log 2^{k-1}\right) \\
&\le cn(\log n)^2 = cn\log^2 n
\end{aligned}$$

$\boldsymbol{T(n) \le cn\log^2 n}$ **이므로,**
$\boldsymbol{O(n\log^2 n)}$**이다.**

최근접 점 쌍 찾기 문제의 응용분야

- 최근접 점 쌍 찾기 문제는 분할 정복 알고리즘과 동시에 대표적인 기하 알고리즘(Geometric Algorithms) 중 하나로, 아래의 분야에서 널리 활용된다.
  - ✓ 컴퓨터 그래픽스 및 컴퓨터 비전
  - ✓ 지리 정보 시스템(Geographic Information System, GIS)
  - ✓ 분자 모델링(Molecular Modeling)
  - ✓ 항공 트래픽 조정(Air Traffic Control) … 등

분할 정복 알고리즘 적용 시 유의할 점

분할 정복 알고리즘 적용 시 유의할 점

- 모든 문제를 분할 정복 알고리즘으로 해결하는 것은 아니다.
- 실제로 분할 정복 알고리즘을 적용했을 때, 계산 비용이 더욱 증가하는 경우도 존재한다.
- 분할 정복 알고리즘이 부적합한 경우는?
  - ➢입력이 분할될 때마다 **분할된 부분 문제의 입력 크기의 합**이 **분할되기 전의 입력 크기보다 커지는 경우**

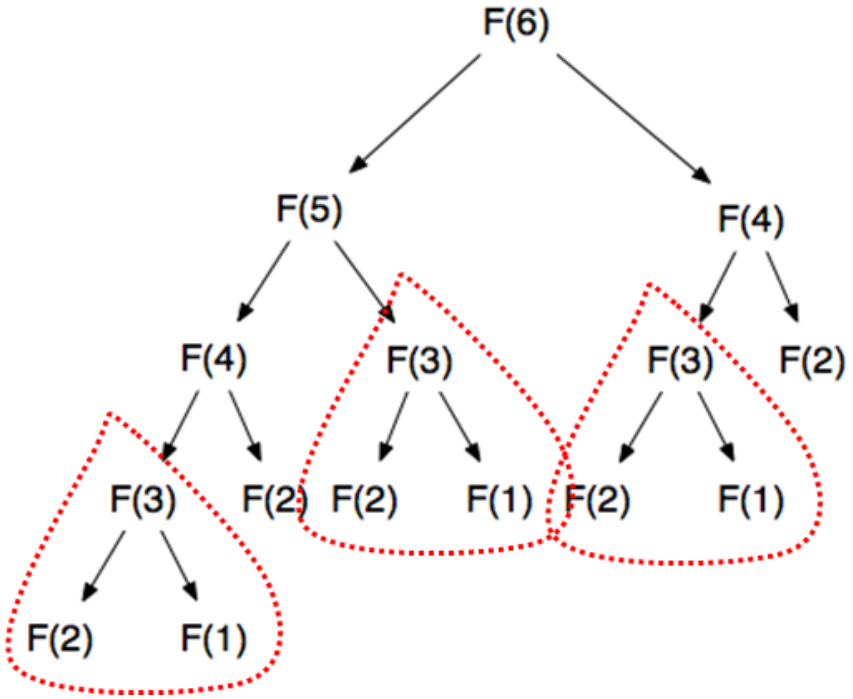
피보나치 수

- n 번째의 피보나치 수 계산하기
  - n번째 피보나치 수 계산을 위한 점화식: $\boldsymbol{F(n) = F(n-1) + F(n-2)}$
  - 위의 점화식은 재귀식으로 정의되므로 순환 호출을 사용하는 것이 자연스러워 보이나, 이 경우의 입력은 1개이지만, **사실상 n의 값 자체가 입력 크기**이다.
  - 즉, 2개의 부분 문제인 $F(n-1)$과 $F(n-2)$의 입력 크기는 $(n-1)+(n-2) = (2n-3)$이 되어서, **분할 후 입력 크기가 거의 2배로 증가**한다.

피보나치 수

- 피보나치 수를 계산하기 위해 부분 문제들로 분할하면…
  - $F(6)$을 계산하기 위해서는 $F(2)$를 **5번 중복**하여 계산해야 하고, $F(3)$은 **3번 중복**하여 계산된다.

```
FibNumber(n):
  1.   F[0]=0
  2.   F[1]=1
  3.   for i=2 to n
  4.     F[i] = F[i-1]+ F[i-2]
```



분할 정복 알고리즘 적용 시 주의할 점

- 주어진 문제를 분할 정복 알고리즘으로 해결하려고 할 때에 주의해야 하는 또 다른 요소는 **취합(정복) 과정**이다.
- 주어진 입력을 분할만 한다고 해서 효율적인 알고리즘이 만들어지는 것은 아니다.
- 기하(geometry)에 관련된 다수의 문제들이 효율적인 분할 정복 알고리즘으로 해결되는데, 이는 기하 문제들의 특성상 취합 과정이 문제 해결에 잘 부합되기 때문이다.

요약

- **분할 정복(Divide-and-Conquer) 알고리즘:** 주어진 문제를 더욱 작은 크기의 부분 문제들로 분할한 후, 이 부분 문제들을 해결(정복)하여 궁극적으로 최종 문제를 해결하는 방식의 알고리즘이다.
- **합병 정렬(Merge sort):** n개의 숫자들을 n/2개씩 2개의 부분 문제로 분할하고, 각각의 부분 문제를 재귀적으로 합병 정렬한 후, 2개의 정렬된 부분을 합병하여 정렬 (정복)한다. 합병 정렬 알고리즘의 평균/최선/최악의 경우 시간 복잡도는 $O(n \log n)$이고, 공간 복잡도는 $O(n)$이다.
- **퀵 정렬(Quick sort):** 피봇 (pivot)을 기준으로 피봇보다 작은 숫자들은 왼편으로, 피봇보다 큰 숫자들은 오른편에 위치하도록 분할하고, 피봇을 그 사이에 놓는다. 퀵 정렬은 분할된 부분 문제들에 대하여서도 위와 동일한 과정을 순환 호출로 수행하여 주어진 숫자들을 빠르게 정렬한다. 퀵 정렬의 최악의 경우 시간 복잡도는 $O(n^2)$, 평균/최선의 경우 시간 복잡도는 $O(n \log n)$ 이다.

요약

- **선택(Selection) 문제:** k번째 작은 수를 찾는 문제로서, 입력에서 퀵 정렬에서와 같이 피봇을 선택하여 피봇보다 작은 부분과 큰 부분으로 분할한 후에 k번째로 작은 수가 포함된 부분을 순환으로 탐색한다. 선택 문제 알고리즘의 시간 복잡도는 $\boldsymbol{O(n)}$ 이다.
- **최근접 점 쌍 (Closest Pair) 문제:** n개의 점들을 1/2로 분할하여 각각의 부분 문제에서 최근접 점의 쌍을 찾고, 2개의 부분 해 중에서 짧은 거리를 가진 점의 쌍을 찾는다. 그 다음에, 두 개의 부분 해를 취합할 때, 중간 영역 안에 있는 점들 중에 최근접 점의 쌍이 있는지도 확인해야 한다. 최근접 점 쌍 문제 알고리즘의 시간 복잡도는 $\boldsymbol{O(n\log^2 n)}$ 이다.
- **분할 정복 알고리즘이 부적합한 경우**는 입력이 분할될 때마다 분할된 부분 문제들의 입력 크기의 합이 분할되기 전의 입력 크기보다 커지는 경우이다.

Q & A